

研究报告 / 技术体系与实验证据

# 面向 GPU 内核优化的 智能体与编译器协同演进

Agent-Compiler Co-Evolution for GPU Kernel Optimization

秦海岩 Haiyan Qin  
北京航空航天大学

2026 年 9 月 24 日更新  
阅读快照 / 源码 677eb444



报告边界 正文所读源码固定为 677eb444fec593294c470dcc5793262ae769fce2。第 8 节另列 2026 年 9 月 24 日的 B300-M4 后继诊断，明确标出较新的编译器提交和验收边界。历史设备实验仍绑定各自的源码、精确目标、Workload、oracle 和计时协议；原有仓库引用指向所读的固定提交。

## 摘要

同一个算子可以有許多 GPU 实现：它们遵循相同的数学公式，却采用不同的分块、线程分工、存储和同步方式。智能体生成内核后，仅凭编译错误或一个延迟值，往往难以判断下一步该修改候选程序，还是该补足编译器的表达与检查能力。open-cake-ir 用带类型的执行计划记录这些硬件选择，在生成代码前给出可定位的诊断；任务语义、正确性检查、设备计时和运行证据则由各自的模块负责。一次优化运行使用固定的编译器，任务反复暴露出的编译器缺口在后继版本中修复，再回到新任务验证。

当前源码声明五家厂商的八个精确目标，179 项语料检查符合预期。NVIDIA、Apple、Hygon 和 MetaX 已有范围各异的结果；AMD 的设备计时仍在校准。同形状的 TinyGEMM2 对照通过了严格数值检查。历史实现曾慢 13–37 倍；后继 B300-M4 诊断已大幅缩小差距。最新固定形状的配对测量中， $B = 64$  本地候选为  $19.904\ \mu\text{s}$ ，快于 CAKE 导出产物的  $21.440\ \mu\text{s}$ ； $B = 16$  仍慢约 1%， $B = 1$  的外部比较未通过波动门限。这些是尚未进入正式 Evaluation 的局部测量。已有证据说明系统能够承载这些任务和局部优化；经验材料能否帮助智能体在另一种硬件上更快找到好实现，还需要预先分组的实机实验。

关键词：GPU 内核；中间表示；静态验证；智能体；实验可复查性；跨硬件优化

## ABSTRACT

open-cake-ir independently implements a CAKE-inspired system in which agents author typed, hardware-explicit schedules, receive localized compiler feedback, and validate candidates against external oracles and target-specific measurements.

A corpus-gated compiler successor is introduced between frozen optimization runs. The implementation declares eight exact targets across five vendors; device correctness and measurement coverage remain uneven. A same-shape TinyGEMM2 comparison passes strict numerical checks. Later B300-M4 diagnostics substantially narrowed the historical gap. On the fixed  $B = 64$  shape, a committed candidate measured  $19.904\ \mu\text{s}$  versus  $21.440\ \mu\text{s}$  for the CAKE export, a 1.079-times paired lead. The  $B = 16$  candidate remains about 1% slower; the  $B = 1$  external comparison fails its variability gate. These successor measurements are not formal Evaluation results. Cross-hardware knowledge-transfer effects remain unmeasured. The primary software snapshot is commit 677eb444; each measurement retains its own source and assay boundary.

## 阅读约定

阅读结果时，请先看它经过哪一层检查。语料门说明编译器符合已审查的软件预期；设备正确性与计时结果还要说明目标型号、任务、运行版本和测量质量。比较两种智能体编写环境的效果，则需要预先分组的独立运行和同一套确认规则。本文把历史实验、当前仓库记录与 CAKE 论文报告的数字分别标明。

## 内容导览

|                                                 |           |                                        |           |
|-------------------------------------------------|-----------|----------------------------------------|-----------|
| <b>1 问题、研究对象与论点</b>                             | <b>3</b>  | 6.3 研究对照的单位和参考访问 . . . . .             | 13        |
| 1.1 背景：公式、执行计划与智能体反馈                            | 3         | 6.4 证据保存、回放与结论资格 . . . . .             | 13        |
| 1.2 CAKE 论文提供的研究起点 . . . . .                    | 3         | 6.5 结论阶梯 . . . . .                     | 13        |
| 1.3 本工作的定位与当前贡献 . . . . .                       | 3         |                                        |           |
| <b>2 设计原则与实现边界</b>                              | <b>4</b>  | <b>7 实现覆盖与代表性设备案例</b>                  | <b>13</b> |
| <b>3 系统合同与职责边界</b>                              | <b>4</b>  | 7.1 平台覆盖是分层事实 . . . . .                | 13        |
| 3.1 从任务到可审计结论 . . . . .                         | 4         | 7.2 NVIDIA：任务族覆盖与外部比较 . .              | 15        |
| 3.2 编译器：检查程序，再生成目标源码                            | 5         | 7.3 Apple、Hygon、AMD：不同结论等级             | 15        |
| 3.3 实验系统：固定一次优化的条件 . . .                        | 5         | 7.4 MetaX：单内核、完整输出和实际                  |           |
| 3.4 评测与证据：先判对，再解释速度 .                           | 6         | Run 分别验收 . . . . .                     | 15        |
| 3.5 两个时钟：候选搜索与编译器演进 .                           | 6         | 7.5 案例 A：C550 的局部 tiling 收益 . . .      | 16        |
| <b>4 程序表示：任务、完整程序与执行计划</b>                      | <b>6</b>  | 7.6 案例 B：B300 的对齐条件与质量门 .              | 16        |
| 4.1 显式的计划，而非新的布局代数 . . .                        | 6         | 7.7 两个局部案例的结论范围 . . . . .              | 16        |
| 4.2 受限 Python 前端与同一规范 IR . . .                  | 6         | 7.8 历史 B200 案例的放置方式 . . . . .          | 16        |
| 4.3 从 Python Schedule 到 Triton：一次实际生成 . . . . . | 7         | <b>8 与 CAKE 对照：相同的问题，不同的证据</b>         | <b>16</b> |
| 4.4 一个数值反例：FMA 与乘加 . . . . .                    | 9         | 8.1 TinyGEMM2：相同任务上的性能差距               | 17        |
| 4.5 完整 Program 的阶段与绑定 . . . . .                 | 9         | 8.2 KDA prefill：B300 上的有界机制诊断          | 19        |
| 4.6 精确目标文档 . . . . .                            | 10        | <b>9 跨硬件复用：从一次改写到可检验的收益</b>            | <b>23</b> |
| 4.7 表示变更的验收义务 . . . . .                         | 10        | 9.1 可复用的是机制，参数要在目标卡                    |           |
| <b>5 编译器：从局部化检查到目标源码</b>                        | <b>10</b> | 重选 . . . . .                           | 23        |
| 5.1 分层拒绝与反馈 . . . . .                           | 10        | 9.2 E/P 四组实验 . . . . .                 | 23        |
| 5.2 源码生成与下游工具链 . . . . .                        | 11        | 9.3 从 B300 到 Hygon：可执行与性能的分界 . . . . . | 23        |
| 5.3 Target 与生成路线如何保持分离 . . .                    | 11        | <b>10 限制、可复查性与结论</b>                   | <b>24</b> |
| 5.4 从 Finding 到物理资源观察 . . . . .                 | 11        | 10.1 解释结果时的四项限制 . . . . .              | 24        |
| 5.5 Corpus Gate 的含义 . . . . .                   | 12        | 10.2 下一步的可检验目标 . . . . .               | 25        |
| <b>6 实验方法与证据等级</b>                              | <b>12</b> | <b>A 术语、源码入口与完整示例</b>                  | <b>26</b> |
| 6.1 一次 Run 的四阶段 . . . . .                       | 12        | <b>B 结果来源与历史身份</b>                     | <b>27</b> |
| 6.2 双时间尺度与 Finding 的负责位置 . .                    | 12        | <b>C 复现边界与报告构建</b>                     | <b>27</b> |

## 1 问题、研究对象与论点

以按行计算的 Softmax 为例，公式确定了每行应得到什么结果，却没有规定一个 GPU 程序处理多少行、由哪些线程完成归约，也没有规定中间值放在哪里。智能体可以尝试多种安排；其中有的地址或同步关系不成立，有的虽然算对，却比原实现慢。要继续改进，智能体需要知道具体是哪项选择出了问题，而不只是收到“编译失败”或一个延迟数字。

本报告研究的是这样一套编写环境：智能体修改明确的执行计划，编译器在生成代码前检查可判断的关系，设备评测再回答结果是否正确、性能是否改善。当任务反复暴露出表达或分析缺口时，维护者改进编译器，并在后续任务中检验这项改动。贯穿全文的问题是：

**核心问题** 怎样在固定任务和可追溯的实验条件下，让智能体的 GPU 执行选择可以被检查、失败可以被定位，并将反复出现的问题转化为后续可用的编译能力？

CAKE 论文提出了编译器与智能体协同改进内核的研究路线 [1]。open-cake-ir 沿着这一问题独立实现了程序表示、编译检查和实验流程，并把不同硬件上的观察保留在各自的任务与测量范围内 [6]。

### 1.1 背景：公式、执行计划与智能体反馈

数学公式与设备程序之间隔着一层执行安排。以内核中的行归约为例，行块大小改变了每个程序实例承担的工作；线程分工影响归约时的通信；中间值的存放方式又影响寄存器与共享内存压力。编译器可以检查一部分地址、数据流和同步关系，物理资源分配与速度则要到编译产物和设备上观察。因此，一个候选的反馈需要指出检查到了哪一层，以及问题落在哪个操作或资源上。

这些观察还决定改动应当留在哪个位置。某种分块在当前形状上更快，可以先保留为任务候选；换到另一目标时，需要重新选参数并验证；若同类改写在不同任务中反复有效，才有理由提炼带适用条件的编译器变换。报告将依次展示这三层工作所依据的程序、诊断和设备记录。

### 1.2 CAKE 论文提供的研究起点

CAKE 让智能体编写一份带类型的硬件执行计划。计划写明资源、线程角色、数据访问和同步；屏障地址与流水线阶段位等机械细节由编译器推导。编译器先检查程序安全、硬件约束、数据一致性和调度语义，再把有希望的候选送去编译与设备评测。任务中反复出现的失败，可以推动后续增加原语、改进检查规则或校准性能估计。CAKE 的重点因此不只是生成更快的内核，也是让编译环境随着任务一起改进 [1]。

论文在 B200 上用固定形状的 Flash-KMeans 任务比较了两种编写环境：一组由智能体编写 Cake IR，另一组直接编写 CUDA/PTX；目标的低层实现都不向作者开放。两组各运行三次，共同使用 80M provider-token 截点。这个实验比较的是整套编写环境，包括表示、检查和反馈，并未单独测量 IR 语法的作用。论文中的 KDA 服务及其他性能数字也属于原研究的结果；本文讨论本仓库时，会另列其独立运行的证据。CAKE 论文覆盖的目标是 NVIDIA Ampere 至 Blackwell，非 NVIDIA 平台的实现和测量需要另建证据 [1, 6]。

### 1.3 本工作的定位与当前贡献

open-cake-ir 将 CAKE 提出的研究问题落到一套可独立检查的开源实现上。编译器单独接受完整程序及其内核计划，按精确目标给出诊断与生成源码；实验系统负责固定任务、作者材料和预算，评

测与证据模块保留设备结果及运行过程。这种分工让一次内核优化的结果有出处，也让编译器的后继改动能够回到触发它的任务接受验证。

仓库目前支持五家厂商的八个精确目标，并在若干目标上保留了正确性、计时或完整程序的有界案例。本文的贡献是把这些程序表示、检查机制和实验证据放在同一条可读的路径上，说明各目标已经走到哪一步，以及与 CAKE 公开产物重叠的任务还差在哪里。跨硬件经验是否能额外降低智能体的搜索成本，是这一基础之上的下一项实验问题。

## 2 设计原则与实现边界

CAKE 在附录 B.1 给出八项 IR 设计原则[1]。本仓库沿用这些原则来约束新的执行计划语法、检查规则和后端能力。表1列出它们在实现中的具体落点。

表1 P1–P8 对本仓库的实际约束。具体语法和目标准入仍以源码合同为准。

| 原则      | 本仓库的实现落点                               |
|---------|----------------------------------------|
| P1 易于编写 | Python 前端提供变量、表达式和角色作用域；网格细节由计划和编译器推导。 |
| P2 性能透明 | 影响性能的分块、执行组、存储、访问和同步选择保留在 Schedule 中。  |
| P3 规范形式 | Python 与 JSON 进入同一套类型和操作模型。            |
| P4 静态类型 | 类型、操作数与形状错误在构造和检查阶段定位。                 |
| P5 可分析  | IR 显式保留 Verifier 与资源分析需要的数据和执行关系。      |
| P6 语料门  | 新语义同时接受既有正例与反例的 Corpus Gate 检查。        |
| P7 分析一致 | 数据模型改动同时更新对应的合法性与资源分析。                 |
| P8 硬件根据 | 目标指令、存储和同步承诺由 Target 与后端说明其适用范围。       |

硬件事实由精确目标文档声明。遇到未建模的资源上限，系统明确报告覆盖缺口；遇到目标不支持的指令，后端给出拒绝原因。这样，一份计划不会因为名称相近而悄悄降到另一架构。静态检查负责在已建模的范围内提前发现问题，真实输出与速度仍由设备评测确认。

编译器还要保持独立。它解释程序和目标，却不读取研究分组、作者工具或证据库。一次运行固定编译器与任务，运行中比较候选；编译器自身的修改在后继提交通过语料检查后，才用于下一轮任务[2, 5]。

## 3 系统合同与职责边界

### 3.1 从任务到可审计结论

一次内核优化从任务合同（Workload Contract）开始。它规定输入、输出、数学定义和数值容差，给所有候选提供同一个判题标准。智能体提交的执行计划（Schedule）则写明这个内核怎样分工、读取、计算和同步；需要多个内核时，完整程序（Program）再说明阶段顺序及张量如何交接。

运行规格（RunSpecification）把这次尝试使用的源码、目标硬件、可见材料和预算固定下来。如果目的是比较两种编写条件，研究计划（StudyPlan）还要在运行前分配组别、终点和分析规则。表2列出这些事实各由谁负责；有了这样的分工，读者才能从一次候选追溯到它实际参加了哪项任务与评测[2, 15]。

**表 2** 从任务到结论的负责位置。每一行给下一阶段提供事实或产物。

| 对象或模块      | 拥有的事实                    | 对外给出的产物                 |
|------------|--------------------------|-------------------------|
| Workload   | 数学、形状、输入分布、外部答案          | 不随候选改变的验收合同             |
| Compiler   | IR 类型、Target、合法性、后端准入与生成 | Assessment、Finding、目标源码 |
| Lab        | 材料、候选搜索、预算与 Run 控制       | 已提交候选和过程事件              |
| Evaluation | 外部正确性、目标计时质量和剖析          | 独立的检查、计时和 profiler 收据   |
| Evidence   | 原始文件、事件、回放与审计            | 可由原始材料重建的报告             |
| Study      | 处理分配、估计量和缺失策略            | 限定研究问题的分析结果             |

图1展示了典型运行路径。实验系统调用编译器检查候选，再把可运行产物交给评测；评测与证据记录一起返回下一轮。编译器也可以脱离实验系统单独使用，例如在没有 GPU 的机器上检查 FMA 示例。模块依赖与源码入口见 [架构章节](#)和[职责地图](#)。



**图 1** 一次运行中的候选路径。任务提供外部答案，实验系统管理候选，编译器生成源码，评测和证据模块把可核对的结果送回智能体；上方的研究计划只在需要分组比较时使用。

### 3.2 编译器：检查程序，再生成目标源码

编译器读入完整程序及其各阶段计划，在一个精确目标上检查类型、张量绑定、数据依赖和硬件约束。检查结果把两个问题分开回答：`Assessment.accepted` 说明计划是否符合通用语义规则；`lowering_eligible` 说明所选后端能否把它生成源码。比如，计划本身可以表达一种操作，某个后端却尚未实现对应指令，此时反馈会指出后端能力缺口。

通过检查的计划由后端确定性生成可阅读的源码。编译器中的显式改写也必须给出完整的新候选，或说明为何拒绝改写。这些判断只依赖程序、目标与编译器版本；实验组别和设备测量由后面的模块处理 [2, 3]。

### 3.3 实验系统：固定一次优化的条件

一次运行开始前，实验系统先核对任务、编译器、执行环境、基线和预算，再向编写内核候选的智能体提供获准查看的任务材料。搜索中，智能体提交候选，控制器记录剩余的编译与评测次数。搜索结束时，系统从已有候选中固定一个提名，用独立的新样本确认其正确性与性能；确认结果出现后，提名不再改换。

普通工程运行只需自己的运行规格。研究比较则由研究计划提前分配处理组，并让各组沿用同一执行与评测流程。旧实验使用的 CampaignLock 保留为输入适配层，历史数据仍按产生它的版本解释。这使智能体能看什么、能调用什么工具、何时停止，都在结果出现前确定。

### 3.4 评测与证据：先判对，再解释速度

评测模块先用任务合同生成输入，与外部参考逐项核对输出。候选算对之后，才按目标设备的计时与状态重置约定采集样本，并在需要时运行性能剖析。搜索读数、独立确认和剖析结果各有单独收据。若计时波动超过质量门，记录说明的是测量尚不足以判定快慢；候选的数值正确性仍按先前的检查结果保留。

证据模块保存候选文件、事件顺序、评测收据和终态。独立审计可以在不再调用智能体服务或 GPU 的情况下重建运行过程；研究分析只使用通过审计的结果。对多阶段程序，评测还要核对每次启动的顺序与最终输出，并按实际程序区间计时。单内核的计时收据只能回答那次内核调用的问题[5]。

### 3.5 两个时钟：候选搜索与编译器演进

候选和编译器沿两条时间线变化。在一次运行里，任务、目标、编译器及测量规则保持固定；智能体改变的是执行计划。检查与设备评测指出下一份候选该改哪里，也让不同候选的结果可以在同一条件下比较。

如果多次运行都遇到相同的表达或检查缺口，维护者才在运行之外修改编译器。新版本先经过语料测试与代码审查，再用于下一轮任务。这样，旧运行的结果仍对应旧编译器，新运行才能检验编译器改动有没有解决原来的问题[2, 5]。

## 4 程序表示：任务、完整程序与执行计划

### 4.1 显式的计划，而非新的布局代数

对每一行做 Softmax，任务规定的是结果：若  $m = \max_k x_k$ ，则  $y_j = \exp(x_j - m) / \sum_k \exp(x_k - m)$ 。执行计划要补上公式没有写出的事：每个程序实例处理几行，哪些执行组负责归约，中间结果放在哪里，以及按什么坐标写回。图 2 以仓库示例中的 64 行块展示这些选择。

Schedule 把操作、缓冲区、执行组、访问坐标、循环和同步关系存成带类型的结构。编译器因此可以检查：读取和写回是否指向同一行，两组是否争写同一输出，目标是否接纳所需指令。它要求作者写明具体的存储与访问承诺，却没有另设一套通用布局代数；Triton 等下游工具链继续负责物理布局、寄存器分配和最终指令选择[3, 2]。

当计算需要多个内核时，Program 将各阶段接成一个完整程序，声明它们共享的输入输出、执行顺序和张量绑定。此时验收对象是程序的最终输出与完整调用区间，而非某一个阶段的单独速度。

### 4.2 受限 Python 前端与同一规范 IR

示例把 Python 前端导入为 cake，用它声明 @cake.schedule 和张量参数；函数的第一个形参 lm 是 Schedule 上下文的语法名字：前端识别它的方法调用，不会执行一个运行时的 lm 对象。仓库没有为 lm 规定缩写含义；把形参及函数体中的引用一起改名为 ctx，仍会得到同一份规范 Schedule。lm.program 选择行块，lm.role 指定执行组，角色作用域内的调用描述读取、归约、广播和写回。前端解析受支持的 Python 语法树，并为诊断保留源码位置；它不会运行源文件的导入语句或函数体[3]。下面摘录的是当前 Softmax 示例中的实际操作：

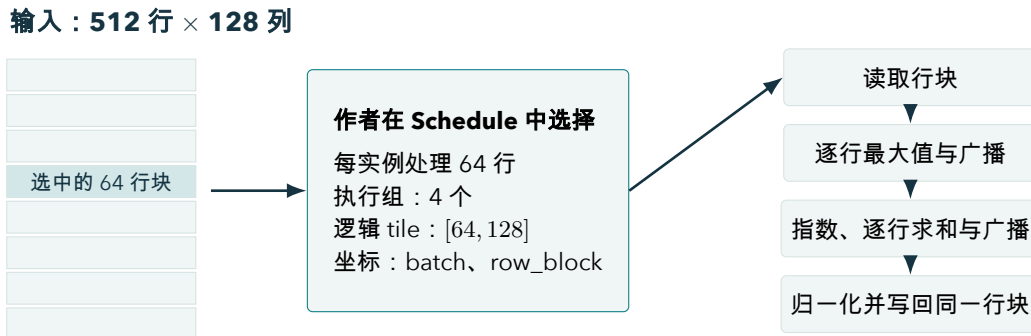


图 2 同一 Softmax 公式对应的一份具体执行计划：每个实例处理 64 行，由四个执行组完成读取、归约和写回。图改绘自[仓库原图](#)，展示的是候选结构，而非计时结果。

```

compute = lm.role(execution_groups=[0, 1, 2, 3])
row_block = lm.program(x, axis=0, dimension=1, tile=64)
batch = lm.program(x, axis=1, dimension=0, tile=1)
with compute:
    x_tile = lm.load(x[batch, row_block, :], reuse="reused", id="load_x")
    rowmax = lm.reduce(x_tile, op="max", axis=1, scope="cta", id="row_max")
    shifted = lm.sub(x_tile, lm.broadcast(rowmax, axis=0), id="shift")
    weights = lm.exp(shifted, id="exponentiate")
    rowsum = lm.reduce(weights, op="sum", axis=1, scope="cta", id="row_sum")
    y_tile = lm.div(weights, lm.broadcast(rowsum, axis=0), id="normalize")
    lm.store(y[batch, row_block, :], y_tile, id="store_y")
  
```

示例输入输出都是 [8, 512, 128] 的 FP32 张量，目标是 sm\_100a，生成路线为 Triton。tile=64 使每个实例负责 64 行；四个执行组完成同一份行块上的工作。行最大值和行和先广播回列维度，再参与逐元素计算；最后仍按原来的行块坐标写回。换一种分块或分工，编译器会重新检查计划，设备评测再判断它是否算对、是否更快。

作者在 Python 装饰器中无需手填任务内容的哈希。独立调用编译器时，可以直接检查计划；进入实验系统后，系统先核对目标、生成路线和公开张量接口，再绑定这次运行冻结的任务。这样，任务身份由实验输入决定，作者只编辑实现方案。具体准入规则见[前端合同](#)。

### 4.3 从 Python Schedule 到 Triton：一次实际生成

对上面的 Softmax，固定源码 677eb444 的 Compiler 先把 Python 语法树转换为带类型的 Schedule，再检查形状、访问、操作依赖和目标约束。本例的结构检查通过，Triton 生成路线可用；报告另给出 REGISTER\_PRESSURE 与 RESIDENCY\_BOUND 两条提示。随后执行 lower，得到下面的 Triton 内核体节选；两处较长的地址表达式只按页宽重新折行。代码中的 CAKE\_OP 注释保留了原计划的操作 ID，便于从生成代码找回 load\_x、row\_max 等编写位置。

对照速览：lm.program 的 64 行块变成程序 ID 与行偏移；lm.load 的访问变成地址、边界掩码和 tl.load；行归约与广播变成 tl.max/tl.sum 和 [:, None]。编译器还把四个执行组写入启动参数 num\_warps=4。

```

row_block = tl.program_id(0)
batch = tl.program_id(1)
row_block_offsets = row_block * BLOCK_ROW_BLOCK + tl.arange(0, BLOCK_ROW_BLOCK)
x_d2_offsets = tl.arange(0, D_X_2)
y_d2_offsets = tl.arange(0, D_Y_2)

# CAKE_OP:load_x
load_x_ptrs = (
    x + batch * N_ROW_BLOCK * D_X_2
    + row_block_offsets[:, None] * D_X_2
    + x_d2_offsets[None, :]
)
x_tile = tl.load(
    load_x_ptrs,
    mask=row_block_offsets[:, None] < N_ROW_BLOCK,
    other=0.0,
    eviction_policy="evict_last",
)
# CAKE_OP:row_max
rowmax = tl.max(x_tile.to(tl.float32), axis=1)
# CAKE_OP:shift
shifted = x_tile - rowmax[:, None]

# CAKE_OP:exponentiate
weights = tl.exp(shifted)

# CAKE_OP:row_sum
rowsum = tl.sum(weights.to(tl.float32), axis=1)
# CAKE_OP:normalize
y_tile = weights / rowsum[:, None]

# CAKE_OP:store_y
tl.store(
    y + batch * D_Y_1 * D_Y_2
    + row_block_offsets[:, None] * D_Y_2
    + y_d2_offsets[None, :],
    y_tile,
    mask=row_block_offsets[:, None] < D_Y_1,
)

```

摘录省略了函数签名和宿主启动包装。完整文件还核对输入的形状、dtype、设备和连续性，随后将  $512/64 = 8$  个行块与 8 个 batch 组成 (8, 8, 1) 网格，传入 BLOCK\_ROW\_BLOCK=64、num\_warps=4 和 maxnreg=96。其中的 tl 是实际导入的 triton.language；前面的 lm 只是 Cake 前端识别的形参名。

读者可在固定源码的根目录运行下列命令，查看包含函数签名、输入检查和启动包装的完整生成文件。这个步骤只生成 Python/Triton 源码，设备编译、正确性和速度另由评测确认。

```

PYTHONPATH=src python3 -m open_cake_ir.cli compiler assess \
    examples/python/softmax.py --format text
PYTHONPATH=src python3 -m open_cake_ir.cli compiler lower \
    examples/python/softmax.py --output /tmp/cake-softmax-generated.py --format text

```

#### 4.4 一个数值反例：FMA 与乘加

FMA 说明为什么公式相同，还要写清计算语义。 $a \times b + c$  可以先乘后加、舍入两次；融合乘加只在最终结果处舍入一次，某些输入下两者会得到不同的 FP32 值。当前 FMA 示例处理  $[8, 128]$  输入，显式请求 `ptx.fma.rn.f32`。图3把任务公式和这份计划的分工放在一起：公式规定应算什么，计划规定八行怎样分配、数据怎样读取和写回[3]。



图 3 FMA 示例的任务与计划。图按仓库原图重绘；高亮行代表一个程序实例要处理的数据，逻辑寄存器 tile 的长度并非每线程物理寄存器数。

#### 4.5 完整 Program 的阶段与绑定

QSA Program v4 是一份可检查的多阶段实例。其任务在  $B = 1, T = 32768$  时读入  $q, k, v$ 、索引查询与索引键，最终写出 BF16 的 output，形状为  $[1, 32768, 32, 128]$ 。Program 中的五个阶段各有一份独立 Schedule；下表直接按合同中的张量绑定与阶段顺序展开，而不是只用“池化、选择、注意力”概括整个程序。

表 3 QSA Program v4 的实际五阶段数据流。形状来自固定  $B = 1, T = 32768$  的任务与 Program 合同；本表描述程序结构，不代表第7节历史候选的计时。

| 阶段         | 读取的张量                     | 写出的张量及作用                                           |
|------------|---------------------------|----------------------------------------------------|
| pool       | index_k                   | pooled: FP32 $[8192, 128]$ ，每 4 个索引键池化一次           |
| layernorm  | pooled, k_norm_weight     | normalized_keys: FP32 $[8192, 128]$                |
| score_topk | index_q, normalized_keys  | block_indices: INT32 $[32768, 512]$ ，每个查询选 512 个块  |
| expand     | block_indices             | token_indices: INT32 $[32768, 2048]$ ，展开为 token 位置 |
| attention  | $q, k, v, token\_indices$ | output: BF16 $[1, 32768, 32, 128]$ ，只读取选中位置        |

例如，合同中的 expand 节点把 score\_topk 产生的块索引接到自己的 Schedule，再写出供 attention 消费的 token 索引。下面保留原 JSON 的数据流字段，省略身份校验字段：

```
{
  "id": "expand",
  "schedule": "corpus/schedules/qa-expand-t32768.json",
  "inputs": ["block_indices"],
  "outputs": ["token_indices"],
  "views": {"block_indices": "identity", "token_indices": "identity"},
  "depends_on": ["score_topk"]
}
```

编译器逐阶段检查 Schedule，也检查 Program 的公共输入输出、中间张量形状与 dtype、生产者和依赖关系；例如 `block_indices` 必须先由 `score_topk` 产生，`expand` 才能读取。合同还声明了输入的 singleton 轴视图，使阶段局部 Buffer 与公开张量对应。五个阶段分别生成目标源码，评测核对最终 output，计时覆盖完整调用序列。此处的 v4 是固定源码中的编译验证实例；第7节的 127.8 ms 历史 QSA 结果绑定它自己的旧版程序与设备收据。

#### 4.6 精确目标文档

同一份执行计划语言要面对不同硬件，具体能力由目标文档（Target）说明。当前八个目标分属 NVIDIA、Apple、AMD、Hygon 和 MetaX；每份文档分别声明线程组宽度、存储空间、可用指令、代码对象与资源限制。编译器按目标的精确 ID 准入，缺少的硬件事实报告为“未建模”。Triton、Metal、原生 CUDA 与 CuTe DSL 则是生成源码的路线；其中 Triton 服务多个厂商，不以厂商名划分公共 IR[4, 2]。

#### 4.7 表示变更的验收义务

新原语会改变作者能写什么，也会改变编译器必须检查什么。因此，一项 IR 扩展要同时说明结果类型、读写效果、合法性规则、分析所需信息和目标上的实际行为，再用正反用例检查这些关系。对于能够由现有原语组合的算子，先建立任务合同和参考实现；遇到确实无法执行的执行行为，再扩展语言与后端[1, 3]。

## 5 编译器：从局部化检查到目标源码

### 5.1 分层拒绝与反馈

编译器先把作者的程序构造成带类型的 IR，再检查程序中的关系是否成立。例如 FMA 需要三个输入；若候选漏掉第三个读取，检查结果会在 `operations[3].reads` 指出 `ELEMENTWISE_ARITY`。作者据此补上缺失的数据边，而不必先编译、运行 GPU 再从错误输出猜原因。其余检查归入四类：

`schedule_semantics` 调度语义

`hardware_conformance` 硬件符合性

`data_consistency` 数据一致性

`program_safety` 程序安全

一条诊断（Finding）带有稳定代码、位置、类别及阻断范围，Python 前端还能把位置指回源文件。建议性提示另存，不改变候选的检查结论。通用规则通过后，后端还会检查这条生成路线能否支持候选所用的操作与硬件特性。通过这些门，意味着候选可以进入下一步编译；设备上的答案与速度仍要分别测量[2, 5]。

## 5.2 源码生成与下游工具链

检查通过后，后端生成一份可阅读的目标源码。选择 Triton 时，这份源码还要交给对应厂商的 Triton 工具链；Metal、原生 CUDA 和 CuTe DSL 各有自己的生成与编译入口。目标文档声明最终代码对象的类别，执行器按该类别加载产物。这样，从执行计划到 GPU 结果的过程可以逐段追踪：源码是否生成、工具链是否编译、设备是否可加载、输出是否算对、计时是否可信，各有自己的记录 [2]。

## 5.3 Target 与生成路线如何保持分离

同一代码生成机制可以服务不同厂商：当前 Triton 路线就接入了 NVIDIA、AMD、Hygon 和 MetaX。厂商身份由精确目标文档声明，后端只说明自己能生成哪类代码对象。表4列出这些关系。新增目标时，先核对它声明的指令、资源和代码对象，再验证所选后端与工具链；添加一个目标名并不会自动带来设备执行资格。

表4 固定源码中的目标与生成路线。代码对象来自各自 Target 文档；设备证据另见第7节。

| Vendor | 精确目标 ID                                                       | 代码对象          | 当前主要源码路线                  |
|--------|---------------------------------------------------------------|---------------|---------------------------|
| NVIDIA | sm_100a, sm_103a                                              | CUBIN         | Triton; 原生 CUDA; CuTe DSL |
| Apple  | apple_gpu_family7,<br>apple_gpu_family8,<br>apple_gpu_family9 | Metal archive | Metal                     |
| AMD    | gfx1151                                                       | HSACO         | Triton / ROCm             |
| Hygon  | gfx938                                                        | HSACO         | Triton / DTK-HCU          |
| MetaX  | xcore1002                                                     | MCFATBIN      | Triton / MACA             |

目标文档还拥有线程束或波前宽度、资源上限和指令事实。某项上限尚未声明时，分析会把它列为未建模，而不是借用另一型号の数値。MetaX 提供的 `triton_arch=80` 只是适配工具链 API 所需的值；物理目标仍是 `xcore1002`。这种区分使报告和代码生成都能说清实际使用了哪台设备 [4, 2]。

## 5.4 从 Finding 到物理资源观察

计划通过静态检查以后，作者还需要了解它在机器上付出了什么代价。Schedule 中声明的逻辑寄存器 `tile`，并不是每线程实际分到的物理寄存器；后端还可能增加暂存、栈与共享内存。工具链编译后的资源报告可在不运行 GPU 的情况下收集。若候选正确但偏慢，目标设备上的性能剖析进一步帮助作者提出关于访存、占用率或工作分配的假设。

这三种反馈各回答一个问题：计划关系是否成立，编译产物用了多少资源，设备执行时发生了什么。它们绑定同一候选的源码和产物身份，方便作者沿着观察逐步修改。分析缺少目标校准时，报告保留未知状态。细节见 [编译资源分析与校准边界](#)。

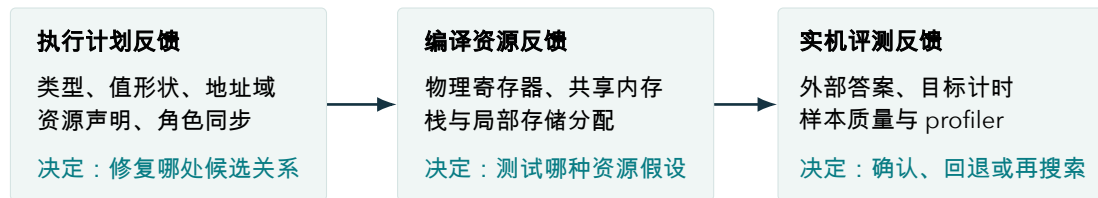


图 4 作者沿着计划、编译产物和设备运行三个层次读取反馈。图按 [仓库既有说明图](#) 改绘；每一层都为下一次修改提供不同的观察。

以下命令可在项目根目录离线检查 FMA 示例，并生成 Triton 源码。它让读者先观察 Assessment 与生成结果；工具链编译、GPU 判对和计时属于后续步骤：

```
PYTHONPATH=src python3 -m open_cake_ir.cli compiler assess \
  examples/python/fma.py --format text
PYTHONPATH=src python3 -m open_cake_ir.cli compiler lower \
  examples/python/fma.py --output /tmp/cake-fma.py --format text
```

## 5.5 Corpus Gate 的含义

当前固定源码 677eb444 的语料门有 **179/179** 项符合预期。它逐项对照已审查的接受或拒绝结果，帮助维护者发现编译器语义变化带来的回归。覆盖报告同时指出两个已声明但尚无该门检查的目标：Apple family7 和 family9。通过数因此只描述这组软件用例；设备运行与性能由各目标自己的实验继续验证 [7, 5]。

## 6 实验方法与证据等级

### 6.1 一次 Run 的四阶段

一次运行从固定任务开始，沿四个阶段推进：

1. 作者提出结构不同的执行计划或完整程序，记录每次改了什么；
2. 编译器先检查类型、合法性和后端支持，再用有目标校准的成本估计安排设备评测顺序；
3. 对可运行候选核对完整输出；正确后按目标设备的协议计时，并在需要时采集剖析数据；
4. 根据具体诊断继续修改候选，或将反复出现的编译器缺口留给后继版本处理。

成本估计节省搜索时的设备时间，最终接受仍取决于外部答案和实机测量。每种目标硬件分别声明计时器、测量区间与样本前的设备状态；缺少这些条件时，报告保留计时覆盖缺口。

### 6.2 双时间尺度与 Finding 的负责位置

图5展示了两种改动的节奏。一次运行只改变候选程序，任务、编译器、目标和测量规则保持原样。运行留下的错误、正确性收据与性能样本，为下一轮候选提供依据。运行结束后，维护者再查看反例：问题是候选写法、检查规则、生成路线，还是缺少一种表达？需要修改编译器时，改动先成为新提交，通过语料门和审查，再进入下一次运行。有些性能差距只适合继续探索候选，记录“暂不提炼通用变换”也是结论。

一份 C550 M17 的实际运行展示了这条路径。在固定源码 896e0887 下，作者进行了十个回合，九份候选被拒绝。第二轮的 ACCESS\_TILE\_MISMATCH 直接指向不一致的访问映射；第六轮



图 5 候选在固定编译器的一次运行中迭代。下方的虚线表示运行结束后，已归档的问题可能推动编译器改动；新版本通过审查后才用于下一轮。

产生的候选后来通过五类输入的独立正确性检查和 MCPTI 成对计时。固定基线与候选的中位数分别为 12.032 和 11.008  $\mu\text{s}$ 。这份记录覆盖了反馈、筛选、确认到终态的过程；计时只对应那次 `local_serialized` 容器环境。详情见[架构案例](#)。

### 6.3 研究对照的单位和参考访问

研究比较以一次独立运行为单位，而不是把同一运行中的多个候选当作重复样本。干净起点允许作者看任务规格、外部答案和高层代码，目标的完整低层实现则留在作者视野之外；研究已知内核的复现时，参考实现可以成为允许的材料。比较 Cake 和直接 CUDA 编写环境，还须固定模型、任务、预算与评测规则。搜索读数用于挑选候选，结论取自候选固定后的独立确认。基础设施中断在报告中记为缺失，保留它已经消耗的成本 [6, 15]。

### 6.4 证据保存、回放与结论资格

每份候选、评测收据和终态都按事件顺序追加保存。审计者可以离线重算候选选择与停止状态，并检查报告是否来自这些原始材料。还要分别核对归档内容能否完整回放，以及当时的写入与发布过程是否满足文件监管要求。前者说明记录可读，后者决定它能否用于晋升或研究估计。协议将两项分别记为 `archive_integrity` 与 `filesystem_custody_verified`。历史运行始终按当时的源码与协议审计 [5]。

### 6.5 结论阶梯

一份结果能支持多强的说法，取决于它实际通过了哪些步骤。表 5 列出从计划检查到研究比较所需的证据，也供后面的平台案例对照。

源码版本由干净的 Git 提交确定：编译器身份写作 `open-cake-ir@<commit>`，执行器还要加上精确目标。本文固定的提交用于说明当前软件；后面的历史测量仍保留各自运行时的提交、样本与监管记录。

## 7 实现覆盖与代表性设备案例

### 7.1 平台覆盖是分层事实

仓库已接入五家厂商，但“能生成代码”“设备算对”和“计时可比较”是不同阶段。表 6 先给出每条路线已经到达的范围；具体数值在各自的任务、设备和基线之内解释。各平台的原始报告与收据仍是结果来源 [2]。

表 5 从计划检查到研究比较，各种结论需要的证据。

| 可陈述的结果        | 必要证据                             | 尚需另行验收        |
|---------------|----------------------------------|---------------|
| 结构与目标准入       | typed IR、精确 Target、Finding 与后端预检 | 工具链编译和加载      |
| 设备正确性         | 完整输出对外部 oracle，覆盖规定形状与分布         | 计时质量和性能收益     |
| 局部性能          | 固定候选与基线、共同计时区间、质量通过              | 跨任务或跨硬件泛化     |
| Program 或服务收益 | 完整调用路径、dispatch 与应用层验收           | Agent 处理的因果效果 |
| 研究归因          | 预注册处理、独立 Run、共同确认与缺失规则           | 未测试任务与平台的外推   |

表 6 截至报告快照的代表性平台能力与主要解释边界。

| 平台     | 已报告的执行范围                                        | 需保留的边界                                  |
|--------|-------------------------------------------------|-----------------------------------------|
| NVIDIA | B300 的单 Kernel、部分完整 Program；B200 另有固定案例         | FlashInfer 外部比较有通过与未通过质量门的边，按任务解读       |
| Apple  | Metal 固定算子与 TaskLab 优化记录                        | 已验收设备和计时口径不推广到全部 family 或完整 Program     |
| AMD    | gfx1151 设备调查与 smoke                             | 计时器绝对时间未对齐，尚无合格的完整 Agent 收益结论           |
| Hygon  | BW1101 / gfx938 固定任务与 HIP 路线                    | 短 Kernel 的计时分辨能力限制部分比较                  |
| MetaX  | C550 固定单 Kernel 及若干完整输出；单 Kernel 配对计时和 profiler | 完整 Program 普通 Run 计时与完整 provider 闭环仍需验收 |

## 7.2 NVIDIA：任务族覆盖与外部比较

在 B300 上，FlashInfer 逐任务综述 为每个固定形状保留原起点、改写产物和外部实现。截至 2026-09-21，26 个任务中有 16 个完成三方数值比较。按综述选定的代表产物，对外部实现的计时结果分为：2 项合格领先、6 项未检出达到预设门槛的差异、4 项合格落后、4 项缺少合格的外部计时；其余 10 项还在完成外部性能比较。这个分布展示任务进度，不能合成为一个平台加速比。

逐任务记录更能说明优化发生了什么。任务 022 的四组候选对外部实现取得合格领先，但它与原起点之间的计时质量不足，原起点仍保留为比较对象。任务 003 的整行改写相对中间候选改善，面对外部实现仍落后。这些结果把“相对自己的上一版进步”和“达到外部实现水平”分成两件事。独立 NCU 剖析可帮助解释工作分配与资源使用；计时判断仍依据原来的 CUPTI 样本及质量门 [8]。

B300 的服务实验、CTA 宽度验证、FlashInfer 改写和 CAKE 任务复现使用不同基线与协议，分别放在报告中。B200 的 FMA、状态操作和 QSA 案例也沿用各自的历史提交。

## 7.3 Apple、Hygon、AMD：不同结论等级

Apple 的 Metal 记录涵盖几台不同设备。M4 上两个固定的 FP32 行归约任务，相对各自的起点保留了 1.107 和 1.138 倍的历史改进；M1 Pro 上的小形状 GEMM+SiLU，相对朴素任务基线确认了 23.604 倍。后一数值主要说明起点留下了很大优化空间，所用基线与专家库实现不同。M2 的 GEMM 搜索虽有较低读数，计时稳定性尚不足以给出性能结论。Metal 的样本覆盖整个 command buffer，并按重复调用均摊；它和 NVIDIA 的 CUPTI 样本各有自己的区间 [9]。

Hygon BW1101 展示了另一种结果组合。逐通道矩任务从 9.439 降到 6.719  $\mu$ s，原记录判为确认加速；prelu 则从 5.439 增至 9.759  $\mu$ s，属于确认变慢。多项短内核的两边都读到 5.439  $\mu$ s，来源把它们标为测量分辨能力待查，而非真正持平。部分历史行没有在汇总表转录完整形状，读者可以沿链接回到原运行查看 [10]。AMD gfx1151 现有 Triton/HSACO 路线和设备调查；两种计时器的绝对时间仍需对齐，因此其结果页暂不列加速比 [11]。

表 7 Apple、Hygon 与 AMD 的代表性观察。每行沿用自己的任务基线和计时资格。

| 目标           | 固定案例          | 观察              | 可陈述的范围           |
|--------------|---------------|-----------------|------------------|
| Apple M4     | 两项 FP32 行归约   | 1.107 / 1.138 倍 | 各自任务的历史晋升记录      |
| Apple M1 Pro | GEMM+SiLU，小形状 | 23.604 倍        | 相对朴素任务基线，非专家库对照  |
| Hygon BW1101 | 逐通道矩          | 1.405 倍         | 首个保留合格 Run 的本机结果 |
| Hygon BW1101 | prelu         | 0.557 倍         | 确认变慢，应保留负结果      |
| AMD gfx1151  | RMSNorm smoke | 延迟缺失            | 路线调查；计时器未对齐      |

## 7.4 MetaX：单内核、完整输出和实际 Run 分别验收

C550 的记录覆盖三件互有关联的工作。第一项是固定形状 M17 GEMM 的单内核分块优化，下节详述它相对本机基线的独立确认。第二项是 GQA、MLA 和 MoE 等 17 个程序变体的完整输出核对，86 个 case 全部通过；这些收据记录的是正确性，没有普通程序计时样本。第三项是一次十轮 Agent Run，保留了候选被拒、修正和最终确认的过程。该次 Run 中 12.032/11.008  $\mu$ s 的配对中位数属于固定基线与 local\_serialized 运行环境。三种观察各自回答实现、判对和作者流程的问题，详见 C550 专题与集成案例。

## 7.5 案例 A : C550 的局部 tiling 收益

### 1.241 倍 C550 · FP16 GEMM · $M = 17, N = 128, K = 2048$

基线  $72.448 \mu\text{s}$  → 候选  $58.368 \mu\text{s}$ ; M tile  $64 \rightarrow 32$ 。独立确认的正确性与测量质量门通过。

这是 `local_serialized` 范围内的本机 authoring 对照，支持这个形状上的局部调度改进。这次只改变 M tile，独立确认说明更小的 tile 对当前形状有利。将这一观察提炼为跨目标经验，还需要在目标设备上重新选择参数并验收收益 [12]。

## 7.6 案例 B : B300 的对齐条件与质量门

### 1.081 倍 B300 · FlashInfer 任务 026 · 守卫式对齐

固定 `generic sliced-w8` 对照  $6.4005 \mu\text{s}$  → 候选  $5.920 \mu\text{s}$ ; 同次候选对照边通过计时质量门。

这项对齐改动对固定的通用实现取得了可确认的局部收益；与外部参考的那组样本没有通过 CV 质量门，因此外部胜负仍未确定。任务 008 则把两阶段 Program 跑通并通过 2550 份数值快照，但候选对内部对照和外部参考的计时都不稳定。它提供了完整程序的正确性记录，性能判断需要新的合格样本 [8]。

## 7.7 两个局部案例的结论范围

两个案例分别说明了分块与对齐可以在固定任务上改进本机候选。它们使用不同硬件、任务和计时协议，因此正文按案例报告，同时保留变慢、差异不足和计时失败的记录。当前 [状态页](#) 也没有汇总全项目已验收的 Study Report；各项结果继续从自己的报告和收据追溯。

## 7.8 历史 B200 案例的放置方式

2026 年 9 月旧版报告还保存了两项 B200 案例。QSA 的  $T = 32768$  完整程序，历史确认候选和同次 CUDA 黑盒参考分别约为 127.8 与 676.4 ms，展示了多阶段计划与程序级评测的一次实际使用。Flash-KMeans 的固定形状配对搜索，每组有三次独立运行；Open-Cake 与直接 CUDA 编写环境的确认终点中位数分别约为 1.512 和 5.342 ms。这项本地实验采用 150k token 和 max 努力级别；CAKE 论文采用另一套 80M-token 协议，第 8 节将并列说明条件。两项结果都作为历史快照引用，完整任务、软件和样本来源列在附录 [6]。

## 8 与 CAKE 对照：相同的问题，不同的证据

本仓库与 CAKE 有共同的研究问题，也有可以直接检查的重叠任务。不过，论文聚合数字、公开的 CAKE 生成产物、本仓库运行记录和某个任务的外部参考，各由不同材料产生。表 8 先比较研究设计与实现范围；下节再看同一 TinyGEMM2 任务上能够进行的数值和速度对照 [1, 6]。

CAKE 论文表 2 以 B200 上固定的 FlashML 基线 (0.938 ms) 为参照，在 80M token 截点报告 Cake IR 三次运行最优候选的中位数为基线的 1.144 倍，直接 CUDA/PTX 为 0.928 倍。这里转录的是论文结果。本仓库历史 Flash-KMeans 实验使用 150k token、不同努力级别和自己的基线，因此表 8 并列的是协议，而不是用两个实验的加速比互相比大小 [1, 6]。

**表 8** CAKE 论文与本仓库在研究问题、目标和已获得证据上的对应关系。论文数字与本地运行分别标明来源。

| 问题       | CAKE 论文 v1                                               | 本仓库当前可读证据                                               |
|----------|----------------------------------------------------------|---------------------------------------------------------|
| 执行计划     | 硬件显式 Cake IR; NVIDIA Ampere-Blackwell                    | 独立 typed Schedule/Program; 五 Vendor、八精确 Target 文档       |
| 静态反馈     | Verifier 四类门与目标校准成本排序                                    | 共享四类诊断和后端预检; 无目标校准时报告覆盖限制                               |
| 作者处理     | B200、固定模型和 scaffold, Cake 对直接 CUDA/PTX, 3+3 次, 80M token | 历史 B200 配对运行有自己的 150k/max 条件; 当前无可与论文对齐的 clean-start 估计 |
| 专家端点     | KDA、TinyGEMM、Alpha-MoE 等生产任务及服务/portfolio 结果             | 部分固定任务复现与完整输出; 大范围 dispatcher、服务及部分原语仍缺                 |
| 非 NVIDIA | 原论文未测目标侧 lowering 与成本                                    | Apple、AMD、Hygon、MetaX 逐目标实现; 正确性和计时资格不均                 |

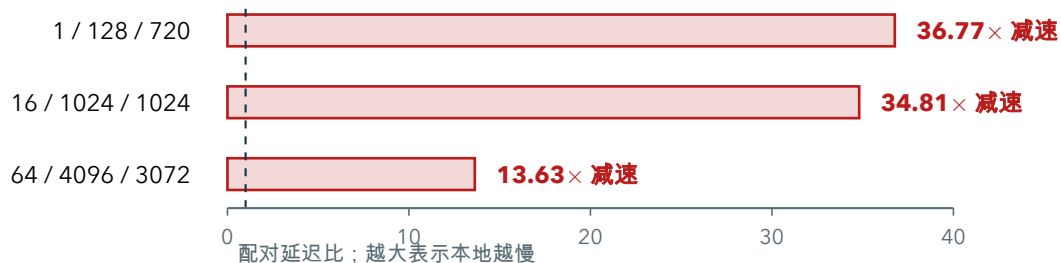
### 8.1 TinyGEMM2 : 相同任务上的性能差距

TinyGEMM2 提供了更具体的对照。仓库固定三个 B300 BF16+bias 形状, 用独立 FP64 数学参考核对结果, 再要求候选与官方导出产物逐位一致。本地候选的 30 次检查和官方导出的 15 次检查全部通过。随后, 五轮交替顺序的 CUPTI 冷 L2 测量保留了两组完整样本。表9列出 stage4 实现的中位数: 本地候选已经重现了任务的数值结果, 速度仍落后于对应的 CAKE 导出产物 [13]。

**表 9** B300 三个固定 TinyGEMM2 形状的历史中位数 ( $\mu s$ )。减速倍数使用原始成对样本计算; 此表比较生成产物, 不是论文的智能体起点实验。

| $B/N/K$      | CAKE 导出 stage4 | 本地 stage4 | 本地相对减速  |
|--------------|----------------|-----------|---------|
| 1/128/720    | 2.720          | 100.192   | 36.77 倍 |
| 16/1024/1024 | 3.040          | 105.825   | 34.81 倍 |
| 64/4096/3072 | 21.216         | 289.186   | 13.63 倍 |

**本地 stage4 相对 CAKE 导出 stage4 的描述性减速**



**图 6** 严格数值检查通过后, 本地 stage4 在三个 B300 固定形状上仍比官方 CAKE 导出 stage4 慢。图示历史配对中位数; 各轮原始样本与质量记录见 [专题](#)。

**2026 年 9 月 24 日 B300-M4 后继诊断。** 表10在同一轮三臂比较中重新测量旧 Triton、后继 Compiler 产物和官方 CAKE 导出产物。后继编译器源码为 4da760be; 前两形状使用 Schedule 声明的原生 CUDA warp MMA 路线,  $K = 3072$  使用按 K 片段直接加载的 Triton 路线。三个形状各有五种输入

分布，后继输出均与导出产物逐位一致，独立数学参考检查通过，输入保持不变。五轮交替顺序测量中，每臂每轮保留 25 个 CUPTI 冷 L2 样本，且每臂只启动一个内核；表中的倍数取五轮配对延迟比的中位数，不能用已四舍五入的表格延迟相除替代。

**表 10** B300-M4 后继诊断，三个固定 TinyGEMM2 形状的延迟中位数 ( $\mu\text{s}$ )。旧到新的提升、后继相对 CAKE 的延迟比均按同轮配对数据计算；这不是正式 Evaluation，也不是 CAKE 论文的智能体起点实验。

| $B/N/K$      | 旧 Triton | 后继     | CAKE 导出 | 旧/后继     | 后继/CAKE  |
|--------------|----------|--------|---------|----------|----------|
| 1/128/720    | 100.256  | 2.784  | 2.624   | 36.045 倍 | 1.061 倍* |
| 16/1024/1024 | 106.048  | 3.424  | 3.008   | 30.972 倍 | 1.138 倍  |
| 64/4096/3072 | 289.123  | 24.672 | 21.408  | 11.722 倍 | 1.151 倍  |

\* B1 的 CAKE 臂最大轮内变异系数为 8.137%，超过预定的 5% 门限；该臂的外部延迟差仅作描述。其余旧/后继配对边均通过此门限，B16/B64 的后继/CAKE 配对边也通过。

已观察到的机制与界限。旧 Triton lowering 将整个  $16 \times 1024$  操作数装入后再对四个  $K = 256$  片段执行 `tl.gather`；按片段直接加载显著缩小了离线生成代码，并在三个形状上消除了多数延迟。对两种短 K 形状，后继 Schedule 进一步明确四个有序的 FP32 部分积、 $K = 64$  的 TMA descriptor 片段、生产者与计算组的同步及 warp 局部 `mma.sync`，保持原有求和与 BF16 写回语义。一个以标量 global-to-shared 拷贝代替 TMA 的有界原型虽然逐位正确，在 B1/B16 上分别达到 19.840/23.360  $\mu\text{s}$ ；改用四个 K64 TMA box 后的短 K 路线显著更快。把  $K = 256$  一次放入单个 descriptor 的尝试则在 `cuTensorMapEncodeTiled` 阶段被拒绝，不能当作可运行的优化。对长 K 形状，保持两个执行组、两级流水；把深度降为一级虽仍逐位正确，在独立配对测量中反而慢 1.538 倍。上述对照支持所选发射方式和分工对这三个形状的改进；缺少可用的动态 profiler 计数器，尚不能把全部收益唯一归因于某一硬件资源。

继续追赶 CAKE 的固定形状复测。表 11 保留了前表之后的两个提交，不能把不同行的延迟相除当作同轮比较。 $B = 64$  的后继按固定形状将输出特征 tile 从 16 增至 32，保持四路  $K = 256$  部分积、两个执行组和两级流水；五种输入均逐位一致。独立复验后，提交 21abdd5d 的产物在五轮配对冷 L2 CUPTI 测量中以 19.904  $\mu\text{s}$  对 21.440  $\mu\text{s}$  稳定快于 CAKE 导出产物。该选择只用于已测的 64/4096/3072 形状，不推广为通用 tile 规则。

两种短 K 形状的原生 emitter 原先把 Schedule 中排在 MMA 前的 bias load 发射到归约之后，使全局读取落在计算关键路径。将其提前并预取 TMA descriptor 后，提交 35f66aca 的十个形状与输入分布组合仍全部逐位一致； $B = 16$  缩到 3.040  $\mu\text{s}$ ，距 CAKE 的 3.008  $\mu\text{s}$  约 1%。 $B = 1$  本地中位数 2.592  $\mu\text{s}$  低于 CAKE 的 2.624  $\mu\text{s}$ ，但两臂最大轮内变异系数分别为 5.557% 和 8.080%，都超过 5% 门限，不构成合格超越。减少 TMA 生产者 warp、改用 timed wait 和一级流水均无稳定收益；交换特征与 batch 的 CTA 方向虽有小幅改善，当前 Schedule 尚不能显式表达对应的寄存器结果与公开输出的转置关系，因此没有将原型偷渡到后端。

**表 11** B300-M4 最新固定形状诊断中位数 ( $\mu\text{s}$ )。每行各自完成五轮交替顺序、每臂每轮 25 个 CUPTI 冷 L2 样本；倍数为同轮 CAKE/本地延迟比的中位数。各行是不同作业，不是一次三形状联合 Evaluation。

| $B/N/K$      | 本地源码     | 本地     | CAKE 导出 | CAKE/本地 | 外部质量  |
|--------------|----------|--------|---------|---------|-------|
| 1/128/720    | 35f66aca | 2.592  | 2.624   | 1.012 倍 | CV 未过 |
| 16/1024/1024 | 35f66aca | 3.040  | 3.008   | 0.979 倍 | 通过    |
| 64/4096/3072 | 21abdd5d | 19.904 | 21.440  | 1.079 倍 | 通过    |

该方向与提前 bias、descriptor 预取合并的  $B = 1$  原型曾测到 2.528 对 2.592  $\mu\text{s}$ ，但本地与 CAKE 两臂最大轮内变异系数分别为 7.925% 和 9.026%，同样不能据此宣称超越。

前一轮原始记录位于 B300-M4 的 gpuq-cd8190e80e06 作业；Finding F-2026-09-24-002 的第 10-13 项索引前一轮与一级流水消融，第 17、22、24 项索引最新的 B64、B16 和 B1 产物及原始报告。截至本次更新，后继代码和 Finding 仍在任务分支，尚未进入维护分支。这次是 broker 准入的定制诊断：B300-M4 的 broker 未提供现有正式 Executor 所要求的 GPUQ\_JOB\_ID，因此尚无正式 Evaluation 收据，不进入 docs/results/nvidia/ 的发布成绩。CAKE 导出产物原有的更大形状集合与服务验证，以及 KDA、Alpha-MoE 的完整任务，仍在后续复现范围之外。

## 8.2 KDA prefill : B300 上的有界机制诊断

仓库已为原始 KDA prefill 六个形状另立精确 sm\_103a 的 B300 Workload，并建立独立 BF16 逐 token 递推 oracle；原有 B200 任务及其成绩未作改写。固定 H64、D128、T8192 形状上，仅将原始 CAKE 导出物的目标检查和编译架构适配到 B300 后，完整输出和最终状态均通过 atol = rtol = 0.01 检查。此适配参考的单内核、五轮 CUPTI 冷 L2 诊断中位数约为 456  $\mu\text{s}$ ，不能称为原始 B200 成绩。

**表 12** B300-M4 的 KDA prefill H64/T8192 有界同机对照。每行的参考与原型在同一 broker 独占作业内交替计时五轮；数字为各自的中位数 ( $\mu\text{s}$ )，倍数为轮内成对比值的中位数。原型均未获 Cake Schedule 准入，也不是正式 Evaluation。

| 原型                      | 适配 CAKE 参考 | 原型         | 原型/参考    |
|-------------------------|------------|------------|----------|
| native CUDA 逐 token 标量  | 456.195    | 16,979.920 | 37.221×  |
| Triton chunk32、V tile64 | 456.035    | 96,080.520 | 210.687× |
| Triton chunk16、V tile32 | 456.259    | 83,264.993 | 182.495× |

三种原型在每轮均通过全部 67,108,864 个输出和 1,048,576 个最终状态元素的独立检查。CPU 上的 chunk32 代数及 BF16 矩阵操作数模拟也通过完整输入。标量 CUDA 虽无寄存器溢出，却逐 token 更新大状态；首个 Triton chunk 映射产生每线程 176 字节栈空间，缩小 tile 后静态栈与本地访问消失，耗时仍达 83 ms。这些记录表明只换语言或只消除该静态溢出信号都不足以接近适配参考；由于 tile 参数同时变化，无法从两次 Triton 读数单独量化溢出代价。适配参考可见 32-token 分块、TMA、tcgen05 MMA、TMEM 状态驻留和分工流水线，而当前 native/CuTe 后端尚不接纳完整的可变状态 Schedule。后继工作须先实现该状态与同步合同，再做同一形状的设备验证；目前没有可称为 CAKE 性能提升的 KDA 结果。完整实验路径由本地 Finding F-2026-09-24-003 索引。

**2026 年 9 月 25 日组件后继。** Cake 生成的 TMEM-A 根收缩、双 chunk 状态传递，以及 BF16  $K = 128$  的 MN-major B 收缩，现均在精确 B300 上通过三组完整输出检查；这三项仍不是完整 KDA。七输出融合预处理也已通过 H64/T8192 的全部元素检查，包含参考值和设备值的有限性检查。其冷 L2 CUPTI 诊断见表 13。

**表 13** B300-M4 固定 H64/T8192 的不同计算范围诊断中位数 ( $\mu\text{s}$ )。各行来自不同作业，未进行成对性能比较；预处理行只产出中间值，不能与完整 prefill 行当作同一算子的加速比。预处理各有五轮、每轮 25 次 CUPTI 冷 L2 样本。

| 范围                 | 中位数     | 当前结论              |
|--------------------|---------|-------------------|
| 适配 CAKE 完整 prefill | 456.034 | 单内核完整输出及最终状态通过    |
| 21-MMA 七输出预处理      | 637.606 | 组件正确；单阶段已超过参考完整内核 |
| 2-MMA 七输出预处理       | 389.507 | 组件正确；后续完整配对见下文    |

后继将显式逆矩阵的 18 个 BF16 MMA 改为待实现的逐 token 三角求解：CPU 上以严格下三角 BF16 系数  $P$  计算  $U_t = RHS_t + \sum_{i < t} P_{t,i} U_i$ ，再将  $U$  舍入为 BF16 供输出及状态收缩。T65、T257 和完整 H64/T8192 的独立逐 token oracle 均通过。改动同时把预处理的编译后寄存器数从 227 降到 185、动态 shared 从 32 KiB 降到 24 KiB；组件延迟减少 248.099  $\mu s$ ，但这几项变化共同发生，现有数据不能把收益拆归到单一原因。按不同作业的中位数相减，新的预处理距旧参考完整内核仅余约 66.527  $\mu s$ ；这只是非配对的设计余量，不是端到端收益。Nsight Compute 因 ERR\_NVGPUCTRPERM 无法读取性能计数器，因此尚无硬件计数器归因。此时原生状态消费者尚未完成；后续完整计时见下文。高保留门值下逐 token BF16 状态舍入导致的最终状态失败仍需解决。

完整固定形状的首次成对测量。后继将上述 2-MMA 预处理接到一个尚未获 Cake Schedule 准入的 Triton 持久化状态消费者。后者每个 CTA 持有一个 head 的 64 行状态，依次处理 256 个 chunk，在每个 chunk 内按 32 个 token 顺序求解并作输出、状态修正。B300 上全部 67,108,864 个输出和 1,048,576 个最终状态元素均通过独立 oracle，状态原位别名及不可变输入也保持正确。然而，在同一 broker 独占作业内，五轮交替顺序、每臂每轮 25 次的 CUPTI 冷 L2 成对诊断中，适配 CAKE 参考的轮中位数中位数为 456.195  $\mu s$ ，两阶段候选为 10,199.621  $\mu s$ ；轮内成对延迟比中位数为 22.362 倍，最大轮内变异系数为 0.132%。这是一条正确但过慢的固定形状机制探针，不是 CAKE 的性能追平，也不是正式 Evaluation。当前消费者有 256 次跨 chunk 串行迭代，每次又执行 32 步 token 求解及跨 lane 归约；测量将主要工程问题定位到消费者路径，但没有 NCU 计数器，不能把 10.2 ms 精确归因于某条指令。后继独立原生 CUDA 探针已让每条 V 行的 32 个更新值驻留寄存器，完整 67,108,864 个更新值通过数值检查；NVCC 报告 40 个寄存器、零 spill。但这个独立求解原语的冷 L2 中位数仍为 668.229  $\mu s$ ，单独就超过前述另一作业中的完整参考。它使用合成 RHS，不能与预处理延迟相加冒充完整任务测量。由此可见，仅把消费者换成 CUDA/PTX 仍不够；后续继续验证片上状态传递与完整配对计时。

原生 TMEM 状态传递与 TMA 搬运。独立原生 CUDA 探针起初在设备上超时。分段探针发现其 tcgen05.alloc/dealloc/relinquish 的 sync.aligned 指令只由 warp 0 的 lane 0 执行；改为 warp 0 全部 lane 执行后，一块 H64/T32 输出与状态均通过独立 oracle。再以 TMEM 保存并按需读回 BF16 状态，两块 T64 和完整 256 块 T8192 的原生消费者均通过检查。完整版本的 NVCC 报告 255 个寄存器、零 stack/spill；检查覆盖全部 67,108,864 个输出和 1,048,576 个最终状态元素，失败数为零。

**表 14** B300-M4 固定 H64/T8192 的完整两阶段机制。每行均将同一 2-MMA 预处理接到所列消费者；参考与候选在同一 broker 独占作业中交替测量五轮、每臂每轮 25 次 CUPTI 冷 L2 样本，禁用 CUDA graph。行与行之间是不同作业，不应将两行延迟差直接当作成对加速比。

| 消费者                           | 适配 CAKE 参考 | 两阶段候选      | 候选/参考   |
|-------------------------------|------------|------------|---------|
| Triton 串行求解                   | 456.195    | 10,199.621 | 22.362× |
| native TMEM, 手工 B 搬运          | 456.291    | 15,898.575 | 34.845× |
| native TMEM, TMA B 搬运         | 456.131    | 3,879.034  | 8.503×  |
| native TMEM, TMA B, 双 CTA/M64 | 456.034    | 4,075.256  | 8.937×  |
| native TMEM, 双缓冲 TMA B        | 456.322    | 3,866.203  | 8.472×  |
| native TMEM, 独立 chunk、前块重算    | 456.195    | 3,403.799  | 7.461×  |
| native TMEM, 独立 chunk、前块简化    | 456.258    | 2,407.153  | 5.276×  |
| native TMEM, shared 前缀        | 456.132    | 2,034.351  | 4.461×  |
| native TMEM, 双 token、末块状态     | 456.260    | 1,444.107  | 3.165×  |

八种 native 消费者的结果及原地最终状态在各自完整配对作业后逐轮通过独立 oracle，不可变输入保持不变。手工版本每块由一个 warp 搬运约 26 KiB 的四个 BF16 操作数 tile；TMA 版本改用四次异步张量搬运及同一事务 barrier，其完整候选延迟显著下降。双 CTA/M64 版本将每个 head 的 V 行拆成两个互不重叠的 64 行 tile。最初沿用 M128 的两 warp 行映射使每组的后 16 行出错；按照 M64 的四组 16 行 TMEM 映射改正后，完整 oracle 通过，但在另一作业中测得  $4,075.256\ \mu\text{s}$ ，未优于 M128/TMA 的  $3,879.034\ \mu\text{s}$ 。双缓冲版本使 copy warp 在当前 tile 被消费时预取下一块 B，但另一次配对作业测得  $3,866.203\ \mu\text{s}$ ；与单缓冲版本相差很小，不能把跨作业差值当作精确的成对收益。它们仍保留 256 块顺序递推，以及预处理的全局中间值。独立 chunk 版本则为每个 head/chunk 启动一个 CTA，由前一块从零状态重算本块入口状态。前块简化版本省去零状态下必为零的两次初始投影及不被使用的输出修正，使完整候选进一步降至  $2,407.153\ \mu\text{s}$ 。插桩且通过 oracle 的后继在 head 0 的非首块 CTA 中，将当前块的 128-key 状态更新定位为中位数 12,605 个 clock64 周期，输出写回相关段仅 755 周期；这是插桩 CTA 周期，不是未插桩 CUPTI 延迟。把前缀因子协作搬到 shared 后，完整候选为  $2,034.351\ \mu\text{s}$ 。继续仅在末块计算公共最终状态，并用前块最后两个 token 重建入口状态，得到  $1,444.107\ \mu\text{s}$ ，仍比适配参考慢 3.165 倍。这些对不同原型的读数来自不同配对作业，不能将原型间差值当成直接配对收益。该近似在固定 H64 正常输入通过完整 oracle，但 CPU 上的高保留门值输入明确失败。另以 20 个 T257 正常分布随机种子筛查预处理因子的近似入口状态：只保留前一块最后 1、2、4 个 token 时，完整输出和最终状态分别仅有 1/20、19/20、20/20 组通过；单一固定输入的通过不能推为跨输入正确性，目前尚无可接受的分流 guard 或后备路径。只保留前一块最后一个 token 的 GPU 原型在固定 H64 输入上有 1/67,108,864 个输出元素超出容差（token 3616、head 57、V 行 58），虽最终状态全通过，仍判为错误候选，不予计时。保留前两个 token 的 GPU 原型通过完整 H64 oracle。其第一次独占计时作业在采样前因分配设备上存在另一计算进程而被隔离检查拒绝。后继同作业五轮配对取得约 2,146–2,149  $\mu\text{s}$  的候选轮中位数与约 456  $\mu\text{s}$  的参考轮中位数，且两臂逐轮通过 oracle；但首轮候选的轮内变异系数为 8.263%，超过预设的 5% 稳定性上限，因而这组数字仅是未通过验收的诊断，不能列为有效加速比。同时 staging beta 的后继也因轮内变异系数 9.244% 未通过性能门槛；只在末块计算状态、但仍重算前块全部 token 的版本，其独占作业在采样前被外部计算进程阻断，均不列入表 14。下一步须继续压缩当前块求解和全局中间值，并解决高保留输入的正确性。ERR\_NVGPUCTRPERM 仍使硬件计数器归因不可用，且这些原型尚未获 Cake Schedule 准入或正式 Evaluation 收据。

**2026 年 9 月 26 日的有类型双阶段后继。** 将 BF16 状态驻留在 TMEM，依次执行基础 TMA/MMA、32 项 FP32 前代入求解、BF16 更新写回和校正 TMA/MMA 的 K32/C32 双 chunk 原型，已在 B300-M4 上通过 3 组独立输入的完整最终状态检查：每组 4,096 个 BF16 元素均无超差，最大绝对误差为 0，且输入未改写。它在精确 sm\_103a 上的离线编译使用 119 个寄存器，无栈和 spill。后继 IR 新增单元循环的标量块坐标，native lowering 从三维输入提取 K128/C32 的两个 B 块；该变体通过 179/179 Corpus Gate 和适用的 CPU 合同，并在 B300-M4 上通过 3 组完整最终状态检查，每组 16,384 个 BF16 元素均无超差，最大绝对误差不超过  $7.63 \times 10^{-6}$ 。其离线编译使用 230 个寄存器，无栈和 spill。

再加入带相位等待的 BF16 TMEM 读取，将旧状态按 FP32 衰减前缀缩放，与校正 MMA 结果相加后舍入为下一块 BF16 状态。该两 chunk 后继同样通过 179/179 Corpus Gate 和适用 CPU 合同，并在 3 组输入上通过每组 16,384 个最终状态元素，最大绝对误差不超过  $1.20 \times 10^{-7}$ ，输入未改写。离线编译为 255 个寄存器、无栈和 spill；寄存器占用已很高，后续需测量其实际驻留与耗时。两项 K128 验证只覆盖有界状态递推，尚无完整 KDA 输出、六形状或成对计时结论。

同日的四 MMA 输出后继。在两个 carried pipeline 中分别增加 base-query 投影和 U-output 校正，合成两 chunk Schedule 因而同时产出 BF16 输出与最终状态。首次排列过早将 base-query 的 FP32 结果从 TMEM 搬到寄存器，使它跨越整个顺序求解：精确 B300 离线编译为 255 寄存器、112 字节 stack，并报告 108 字节 spill 读写。仅将该 TMEM 回读移到输出合并前后，仍为 255 寄存器，但 stack 和 spill 均降为 0；这是寄存器活跃区间缩短的实证，不是设备延迟收益。后版在 B300-M4 的 3 组输入上分别检查每组 8,192 个 BF16 输出和 16,384 个 BF16 最终状态，全部 0 超差，输入未改写。进一步用显式 transpose→store 将行拥有的 BF16 tile 直接写为 [chunk, token, V]，没有额外全局转置缓冲；精确 B300 AOT 保持 255 寄存器、0 spill，3 组输入的输出与状态也全部通过。随后把公开 BF16 V 的 [chunk, token, V] 读取显式转为按 V 行拥有的寄存器视图，并把公开 FP32 beta-gate 用于  $RHS = \beta(V - \text{base\_prediction})$ 。这条有界 native 映射直接按行线程读取，不产生全局转置缓冲；额外读取者或不匹配的访问映射会被后端拒绝。固定源码 ac102f0e 的远端隔离完整适用 CPU 套件在标准 umask 0022 下为 2,601 passed、16 skipped，另有 1 项 Apple MLX 实机测试不适用；Corpus Gate 为 179/179。本机同套件曾因磁盘空间不足中断，远端默认 umask 0002 又使私有目录权限检查失败；两者均为保留的环境失败，而非数值结论。精确 B300 AOT 使用 255 寄存器、0 stack/spill。broker 作业 gpuq-c841e1995072 的 3 组输入各检查 8,192 个 BF16 token-major 输出和 16,384 个 BF16 最终状态，全部 0 超差且输入未改写；最大绝对误差分别为  $1.20 \times 10^{-7}$  和  $3.06 \times 10^{-5}$ 。准备阶段的 Q/K/G/prefix 等值仍为合成输入，逐 token BF16 状态舍入、64-head/256-chunk 绑定、完整六形状及与原始 CAKE 的同作业计时都尚未通过；因此这些数值不能解释为完整 KDA 性能提升。

**H64 的标量 head 与四维 TMA 后继。**固定源码 d5a9648a 用一个 ProgramMap 标量 head 坐标指定每个 CTA 的所有权，chunk 仍由循环标量选定；四个 B 输入用 [chunk, head, matrix] 的四维 tensor-map 直接装入二维 shared box，公开 V、beta、初始/最终状态和 token-major 输出也都显式携带 head 地址。错误 head、rank-5、其他 head 数量和遗漏输出所有权均由后端拒绝。离线编译为 255 寄存器、0 stack/spill；Corpus Gate 为 179/179，远端完整适用 CPU 套件为 2,672 passed、16 skipped，另有 1 项 Apple MLX 实机测试取消选择。首次设备作业因链接遗漏 CUDA driver 库，在加载 cubin 前失败；修正同一源码的 CPU-only 链接后，broker 作业 gpuq-ad57c793c89c 完成并释放设备。独立 host oracle 在三组 H64、两 chunk 输入上，每组检查 1,048,576 个 BF16 最终状态及 524,288 个 BF16 token-major 输出，均为 0 超差且输入未改写；最大绝对误差分别为  $6.11 \times 10^{-5}$  和  $3.82 \times 10^{-6}$ 。这只验证地址与有界发射：准备值仍为合成输入，256 chunk、逐 token BF16 舍入、完整六形状和 CUPTI 配对延迟均待完成。

**256 chunk 的固定形状后继。**提交 61892f81 把上述 head 映射扩到 256 次单元 chunk 循环，仍让每个 head 由一个 CTA 持有。它通过 Corpus Gate 179/179 和远端完整适用 CPU 套件 2,674 passed、16 skipped，另有 1 项 Apple MLX 实机测试取消选择；精确 B300 AOT 为 254 寄存器、0 stack/spill。broker 作业 gpuq-1f1423e7d3da 释放 GPU 后，独立 host oracle 对一组固定 H64/T8192 合成输入的 67,108,864 个 BF16 输出及 1,048,576 个 BF16 最终状态逐元素检查，均为 0 超差、无非有限值、输入未改写；最大绝对输出/状态误差为  $3.82 \times 10^{-6} / 6.11 \times 10^{-5}$ 。该结果只证明长循环相位和完整固定形状地址，准备值仍为合成输入，且状态在 chunk 边界舍入。随后单组件独占 CUPTI 诊断一次因 Python 环境缺 CUPTI 包路径而拒绝 event 回退，两次因分配设备上有外部进程而在采样前退出；没有有效延迟数据，不能据此比较原始 CAKE。

## 9 跨硬件复用：从一次改写到可检验的收益

### 9.1 可复用的是机制，参数要在目标卡重选

一次优化既可能发现可复用的做法，也会得到只适合当前设备的参数。比如，融合私有的中间张量可以省去一次写回和重读；能否融合取决于消费者、生命周期与舍入语义。具体的 tile、执行组和流水线深度，则要在目标卡上重新选择。可确定性表达的改写可以成为带前提和拒绝原因的显式 pass；只有一次有效的局部参数选择，仍作为任务经验保留 [14]。

### 9.2 E/P 四组实验

若要知道经验究竟提供了多少帮助，先要把“让智能体读说明”和“让智能体调用现成改写”分开。图7以  $E$  控制额外机制材料，以  $P$  控制显式 pass 的调用权，形成四种条件。四组共享任务、编译器、目标设备、正确性和计时规则；工具权限由运行环境实际隔离。 $P1$  组仍需知道调用的最小接口，所以  $E$  测量的是接口之外的说明和案例。目前软件协议已接入，四组在共同目标与预算下的实机迁移实验尚未进行 [15]。



图7 四组迁移实验的处理条件。 $E$  是额外说明与案例， $P$  是显式变换调用权；所有组使用相同的编译器和设备评测。此图表示实验分组，尚无实机处理效果。

令  $p_{ep}$  为在预注册预算内经新鲜确认、达到本机预定改善阈值的 Run 比例。预定主要对比为

$$\begin{aligned}\Delta_E &= \frac{1}{2}[(p_{10} - p_{00}) + (p_{11} - p_{01})], \\ \Delta_P &= \frac{1}{2}[(p_{01} - p_{00}) + (p_{11} - p_{10})], \\ \Delta_{EP} &= p_{11} - p_{10} - p_{01} + p_{00}.\end{aligned}$$

一次独立运行是重复单位。正式研究按任务等权汇总，保留各任务内四组的配对关系；搜索成本、预算耗尽和基础设施缺失也按事先约定记录。四组全部运行并经共同确认后，才能计算上面三个处理效果。

### 9.3 从 B300 到 Hygon：可执行与性能的分界

迁移专题记录了一次具体尝试。B300 上的 pairwise\_sqdist 候选把 K 维按 256 分块；在固定 FP32 形状上，五类输入检查和 CUPTI 配对计时通过，相对本机基线从 411.7945 降到 53.856  $\mu$ s。将同一数学任务准备给 Hygon BW1101 时，团队保留张量、输入、参考答案和容差，先把旧版 Schedule 的角色表示显式适配到当前 execution\_groups。适配后的候选及同卡起点均通过编译器检查，并由 Hygon Triton 编成 HSACO。

原 bw1100 节点的八卡被其他 GLM 进程占用，实验止于离线准入。另一台 bw1100-1 具有空闲 BW1101，但宿主内核和环境与前者不同；因此先为该节点单独捕获 Executor，再在干净源码

4c9f4cc0 上运行相同合同。五类输入在计时前后均判对；新鲜 HIP 配对确认质量通过，同卡起点/来源  $K=256$  候选为  $517.9645/748.749 \mu s$ ，十对均是起点更快。故该机制能够正确执行，原参数却在此节点产生负迁移。本地串行锁不能排除没有使用同一锁的外部任务；不同 CUPTI/HIP 计时器的绝对时延也不构成跨卡速度比。

在运行下一次设备实验前，冻结两个只改  $K$  tile 的 Hygon 后继：128 和 64。离线编译均报告零 scratch，分别占 252 和 148 个 VGPR；按预定的资源排序只选择  $K=64$  上卡。它相对  $K=256$  的生成源码仅改变分块值与调度身份。在相同五类输入、whole- $K$  起点及 HIP 配对协议下，新鲜独立确认全部判对并通过计时质量门，起点/候选为  $518.524/189.586 \mu s$  (2.735 倍)，十对均是候选更快； $K=128$  没有设备计时。因此， $K$  维分块这一来源机制可以在目标卡重新选参后奏效，但来源参数不可照搬。另在上卡前冻结一个未见的同算子形状  $R=1024, K=512, N=64$ ，只比较已选  $K=64$  与该形状的 canonical whole- $K$  起点，不再试其他 tile。五类输入在首次及新鲜独立确认中均零不匹配、计时质量通过；确认中位数为  $106.232/97.593 \mu s$  (1.0885 倍)，十对均为候选更快。第二个形状提供有限的 Hygon 本机形状泛化证据，但收益远小于  $K=1024$  的 2.735 倍；B300 没有在新形状上的来源性能收据，也没有跨算子结论。这是一例限定形状的人工机制迁移和 Hygon 本机收益，并非控制作者与预算、随机分配材料  $E$  或变换权限  $P$  后得到的 Agent 处理效果。

难点也并非单一后端开关：同一带循环归约的 Schedule 在当前 Metal 路线无法表达；Triton 能向 AMD、Hygon 生成代码，却不保证源参数的资源分配或速度；同 ISA 的第二台 Hygon 宿主仍需独立 Executor；C550 的容器锁边界和 M2 的计时质量分别限制设备主张。可表达性、目标选参、宿主身份、测量与因果归因须分层验收。

## 10 限制、可复查性与结论

Open-Cake-IR 已经把候选生成、静态检查、目标代码生成和设备反馈接成一条可追踪的路径。在当前源码上，179 项语料符合预期；在若干固定 Workload 上，也已有设备正确性与局部性能结果。这些结果回答了“系统能否表达、检查并运行具体候选”。要回答“这一机制能否稳定带来跨任务、跨平台收益”，还需要更完整的目标侧测量和预注册对照实验。

### 10.1 解释结果时的四项限制

与 CAKE 的关系。本文复现的是论文描述的设计问题与实验思路，并独立实现 Compiler。论文公布了系统机制和汇总结果，但缺少把原始编译器、完整 IR 与 Verifier、agent scaffold、clean-start 轨迹及计时细节一起重建的材料。因此，同名任务的对照应从输入合同、参考实现和测量范围逐项核对 [6]。

平台覆盖。五家厂商、八份 Target 文档界定了可声明的目标；每个平台还须单独取得工具链、数值、计时器和 profiler 的运行收据。179 项 Corpus Gate 尚未检查 apple\_gpu\_family7 与 apple\_gpu\_family9。MetaX 的完整输出验证和 AMD 的计时器调查，也分别处于不同的证据阶段 [7, 2]。

性能范围。一个形状的候选优于本机基线，并不意味着它胜过外部专家实现，或能直接用于多形状应用。后者还涉及分派守卫、尾部输入、回退路径和框架开销。历史 QSA 数字覆盖完整 Program 的 GPU 区间；M17 数字覆盖单内核。两者都不能直接换算成模型或服务延迟。

机制归因。工程 Run 和局部消融可以发现问题、改进系统。若要判断作者环境或迁移材料带来了多少增益，还须固定处理条件，进行独立 Run 和共同确认，并说明缺失结果如何计入分析。 $E/P$  四组的实机对照尚待开展。

## 10.2 下一步的可检验目标

接下来的实验先补齐各目标的完整输出验证、计时区间、设备状态重置和测量分辨能力，再把单内核结果接入多阶段 Program 与目标框架。对反复出现的表达或分析缺口，记录 Finding 和反例，在后继提交中补上 Verifier 规则、IR 能力或带前提的编译器变换，并观察它能否解决后续任务。迁移实验则固定来源机制、目标开发集和  $E/P$  四组材料与权限，按各目标自己的本机基线报告确认率、搜索成本和条件性能。这样，代码能力的增长与经验迁移的收益便可以分别检验。

## A 术语、源码入口与完整示例

表 15 文中的术语按负责事实区分。英文名是 API 查找键，表中中文名用于叙事。

| 对象       | 本报告中的固定含义                          |
|----------|------------------------------------|
| Workload | 外部算子语义、输入域、oracle 与数值容差            |
| Schedule | 单内核的操作、资源、执行组、访问与同步承诺              |
| Program  | 公共张量、多个叶子 Schedule 的阶段顺序和绑定        |
| Target   | 精确设备声明；事实未建模时不借相邻型号                |
| Run      | 一次冻结的作者环境、候选、预算与终态                 |
| 作者       | 在一次 Run 中提交候选的人或智能体；可见材料与工具由运行规格限定 |
| Study    | 运行前固定研究处理、分配、估计量和分析规则              |
| Finding  | 可定位、可归属的结构、目标或后端诊断                 |

当前 [Softmax 文件](#) 是第 4 节运行示例的完整可读源码：

```
from open_cake_ir.compiler import frontend as cake

@cake.schedule(name="softmax-b8-smoke", target="sm_100a", backend="triton",
               entry_point="cake_softmax_b8_smoke",
               residency={"ctas_per_multiprocessor": 4,
                         "registers_per_thread": 96})
def softmax(lm, x: cake.Tensor((8, 512, 128), "fp32"),
            y: cake.Tensor((8, 512, 128), "fp32", mode="output")):
    compute = lm.role(execution_groups=[0, 1, 2, 3])
    row_block = lm.program(x, axis=0, dimension=1, tile=64)
    batch = lm.program(x, axis=1, dimension=0, tile=1)
    with compute:
        x_tile = lm.load(x[batch, row_block, :], reuse="reused", id="load_x")
        rowmax = lm.reduce(x_tile, op="max", axis=1, scope="cta", id="row_max")
        shifted = lm.sub(x_tile, lm.broadcast(rowmax, axis=0), id="shift")
        weights = lm.exp(shifted, id="exponentiate")
        rowsum = lm.reduce(weights, op="sum", axis=1, scope="cta", id="row_sum")
        y_tile = lm.div(weights, lm.broadcast(rowsum, axis=0), id="normalize")
        lm.store(y[batch, row_block, :], y_tile, id="store_y")
```

这里的 Python 接口通过 AST 构造与 JSON 相同的规范计划；`Compiler.load()` 默认读取项目的当前配置。要在具体 GPU 上运行，还需匹配目标工具链，并完成正确性与性能验收。[Python 前端文档](#)列出了支持的语法及源码位置诊断。

## B 结果来源与历史身份

表 16 正文各类结果的第一阅读入口。报告页面是带范围的投影，原始运行收据继续拥有测量与监管事实。

| 范围               | 固定来源或索引                                                                                |
|------------------|----------------------------------------------------------------------------------------|
| 当前 Compiler      | <a href="#">状态页</a> 、 <a href="#">版本清单</a> 与精确目标文档                                     |
| B300 FlashInfer  | <a href="#">逐任务状态</a> ；每行链接自己的 Finding 与收据                                             |
| B300 TinyGEMM2   | <a href="#">历史严格数值与成对计时报告</a> ；后继 B300-M4 诊断索引 F-2026-09-24-002（尚未正式发布）                |
| B300 KDA prefill | 精确目标 Workload <code>cake-kda-prefill-b300-v1</code> ；有界外部诊断索引 F-2026-09-24-003（尚未正式发布） |
| Apple、Hygon、AMD  | 各平台 <a href="#">发布汇总</a> 与原记录                                                          |
| MetaX C550       | <a href="#">平台专题</a> 及 <a href="#">当前 Run 案例</a>                                       |
| 迁移离线试点           | <a href="#">机制与证据边界</a>                                                                |
| 历史案例             | QSA 与 Flash-KMeans： <a href="#">2026-W36.1 旧报告及其证据附录</a> ；退役路径按 history 标签回放           |

历史 Flash-KMeans 的六次终点属于固定  $B = 32, N = 65536, K = 1024, D = 128$ 、BF16 输入的 B200 Workload，三对延迟（Open-Cake/CUDA，ms）为 1.191845/3.898164、1.511927/7.130795 和 1.512163/5.342292；两组各取自己的中位数，范围不是置信区间。历史 QSA 的  $T = 32768$  完整 Program 候选和同次参考分别为 127.821446/676.423598 ms；该时间不含 Python 包装开销。这里只转录旧报告的有界结论与身份，不以新版 Compiler 重释旧原始记录。

## C 复现边界与报告构建

本文的数字来自已存在的报告、Finding 与原始收据定位；写作和 TeX 编译没有启动新的 provider、GPU Campaign 或计时。阅读者可从固定源码检出运行离线入口，看到 IR 构造、Assessment 与 lowering；设备结果则须在原任务的目标、工具链、参考访问和监管环境下重放。报告源文件位于 `docs/open-cake-ir-technical-report.tex`，在装有 XeLaTeX、中文字体和本仓库已有图像的环境中可直接编译。历史示意图保留其原始来源；其文字表达的能力范围按正文当前源码重新核对，不用旧图中的版本值替代当前状态。

## 阅读与引用说明

本文是项目维护者撰写的仓库技术报告，无 DOI、期刊或同行评审声明。引用设计时请固定本报告及其所读源码提交；引用性能时还需追溯案例自己的执行源码、精确目标、Workload、baseline、oracle、计时区间与质量门。以下文献中的仓库链接均固定到本文所读的 `677eb444`，而后继 TinyGEMM2 诊断由正文所列 `4da760be` 与 B300-M4 原始记录另行定位。正式英文引文题名沿用仓库 `CITATION.cff` 的 `preferred-citation`；封面的中文题名概括本文的研究对象。

## 参考文献

- [1] Z. Ye et al. *CAKE: Compiler-Agent Co-Design for Frontier Kernel Evolution*. arXiv:2608.12629v1, 2026. <https://arxiv.org/html/2608.12629v1>.
- [2] H. Qin. 系统架构与平台能力. open-cake-ir@677eb444, 2026.
- [3] H. Qin. IR 指南. open-cake-ir@677eb444, 2026.
- [4] H. Qin. Compiler 清单与 Target 文档示例. open-cake-ir@677eb444, 2026.
- [5] H. Qin. 验收门. open-cake-ir@677eb444, 2026.
- [6] H. Qin. 论文与本地证据合同. open-cake-ir@677eb444, 2026.
- [7] H. Qin. 当前状态. open-cake-ir@677eb444, 2026.
- [8] H. Qin. B300 FlashInfer 任务综述. open-cake-ir@677eb444, 2026.
- [9] H. Qin. Apple 平台结果. open-cake-ir@677eb444, 2026.
- [10] H. Qin. Hygon DCU 平台结果. open-cake-ir@677eb444, 2026.
- [11] H. Qin. AMD 平台结果. open-cake-ir@677eb444, 2026.
- [12] H. Qin. C550 专题与 M17 独立确认. open-cake-ir@677eb444, 2026.
- [13] H. Qin. B300 CAKE 重叠任务对照. open-cake-ir@677eb444, 2026.
- [14] H. Qin. 跨硬件优化知识迁移机制. open-cake-ir@677eb444, 2026.
- [15] H. Qin. E/P 消融实验设计. open-cake-ir@677eb444, 2026.